

Class Project Phase IV

CSE 360

Team Number: 1

Team Members: John Bostater, Tristan Andrade, Austin Mayhew, Ryan Clark, Juan Rosas Jr.

- a) Link for a the technical presentation (around 15 minutes). You can record this as a YouTube video or upload your video to the google drive of your team leader or one of your team members and provide a link to the google drive video

Google drive or YouTube link here:

1. **An overview of the project**

a.

2. **A summary of Phase I** - explain the use cases and the use case diagram

a. Austin

3. **A summary of Phase II** – explain the class diagram and design

a. Tristan

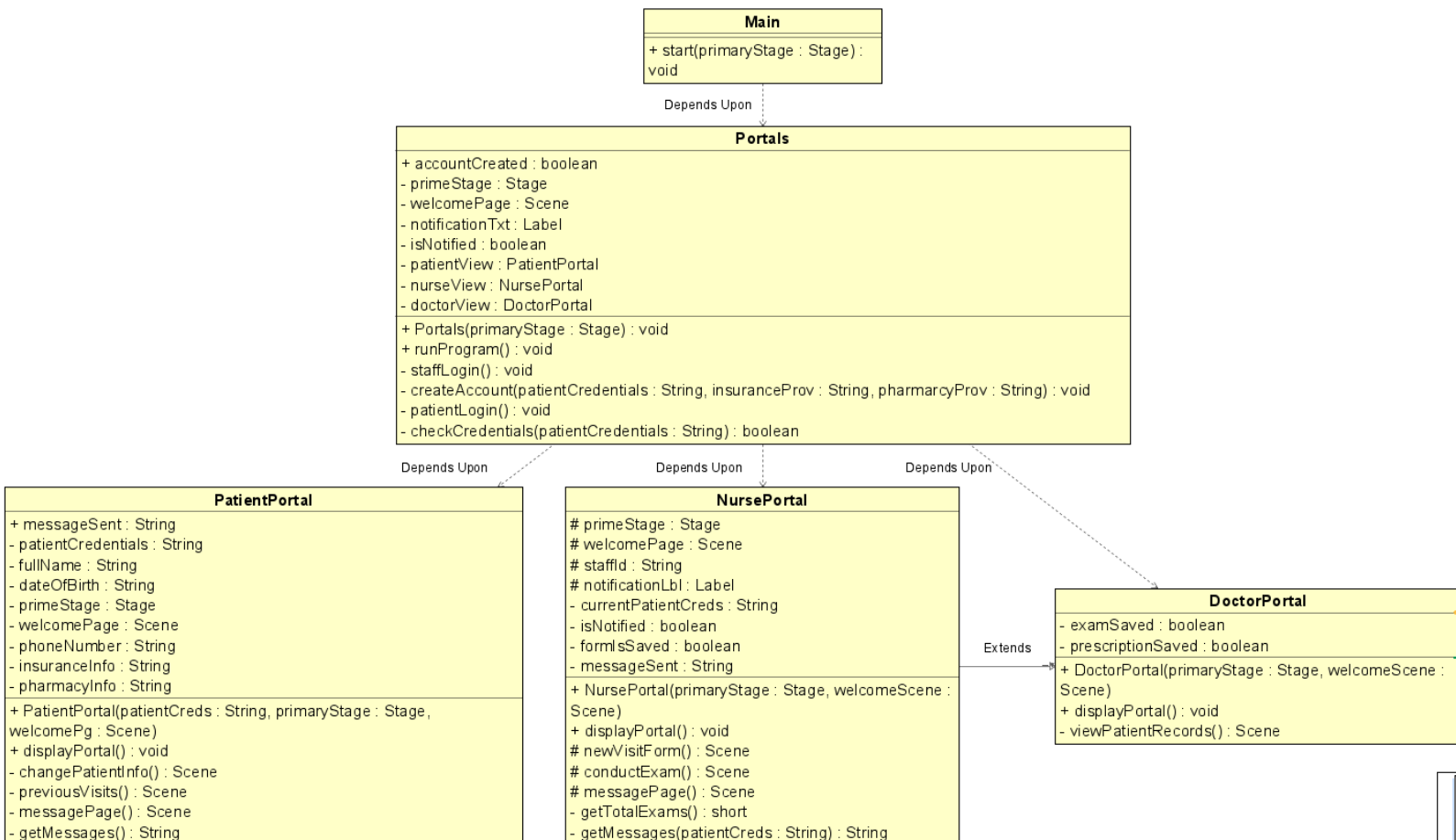
4. **A summary of Phase III** – explain main code modules

a. John

5. **Conclusions** – project summary and lessons learned

a.

- b) Final class diagram (copy and paste the Astah class diagram here).



// TODO: Modify class diagram to include classes for UI/UI components, data, and control logic.
Add more details and refinement to class diagrams.

c) Testing report. Use the following format

Test case	Testing Scenario from Phase II	System Output	Test case passed (Yes/No)
1	1. Nurse clicks “New Visit Form” via the Nurse Portal. 2. Nurse takes Patient’s vitals. 3. Nurse fills out the rest of the form. - Patient’s first name - Patient’s last name - Patient’s birthday	1. A blank New Visit form is opened. 2. Results are added to the vitals section of the form. 3. Form requires that a patient be selected from the dropdown menu. Doing so will clear the form, so it must be done first. Selecting a patient from the dropdown allows the form to be stored in the database.	Yes
2	1. Doctor clicks “Patient Visit Form” via the Doctor Portal. 2. Doctor clicks on “Conduct Exam.” 3. Doctor saves exam results after filling out the form. - Exam results - Notes	1. Visit form scene is loaded, with a dropdown to select a patient and load their saved visit form. 2. Exam scene is loaded, with fields for notes and prescription. Patient’s summary is instead available on the visit form. 3. Results are appended to the visit summary file and are made available to view.	Yes
3	1. Patient clicks on “View Messages” in the Messages window. 2. Patient clicks on Nurse’s message.	1. Messages scene is loaded for the patient. 2. Patient cannot click on a message. Messages are always loaded in their entirety, so this is unnecessary.	Yes

4	1. Patient clicks on “Send Message” in the Messages window. 2. Patient clicks “Send” to send message to Doctor/Nurse.	1. Unnecessary. Text box is always present on the messages scene. 2. Patient’s message is sent and stored in the patient’s messages file.	Yes
5	1. Doctor clicks on Patient’s file. 2. Doctor calls Patient via the phone number on file.	1. Doctor can select a patient in View Patient Records and Messages. In View Patient Records, the visits dropdown will be populated with the patient's previous visits. Visits can be selected, loading the data from that visit. In Messages, the messages from the patient will be loaded, along with their phone number on file. 2. Doctor can view the patient’s phone number and call them using the number.	Yes

d) Conclusions (No more than ½ page, single space 12 font size)

As our project concluded there were many take-aways from our concerted efforts.

The utilization of team organization methods and data structures in our programming process allowed the development of an efficient (almost) bug-free final program. Something that worked well for our team to stay organized was using Scrum practices to have daily meetings for the day’s tasks. Having these meetings helped our team to ensure that at the end of every day, our program was still functioning with no non-critical errors.

While efficiency and a working product were the collective goals, personal obligations among team members prevented many necessary discussions about enhancements to the program and data trimming. In the latter half of the development process, at least **9 bugs** were identified. Out of the 9 bugs, only **1 non-critical bug remained unsolved**, that being the Null Pointer exception in the Doctor’s “View Patient Visits”.

On average our team produced about **988 lines** of code per Object/.java file. Our team is satisfied with achieving an average below 1000 lines of code, but a smaller size was desired. The final program (including images) came out to a total size of **266.3 Kilobytes (KB)**.

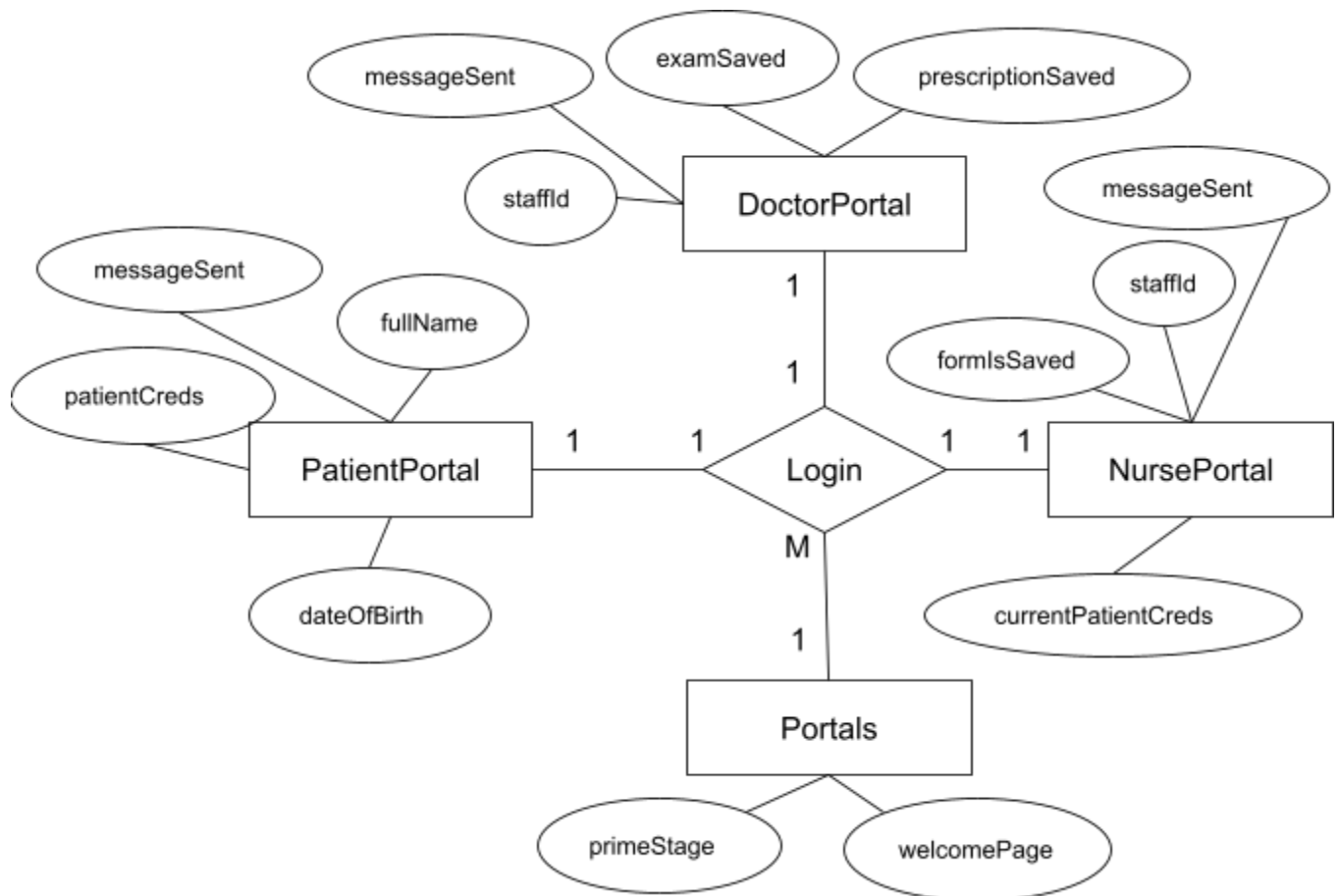
For any future CSE-360 groups developing their software, our team recommends daily communication, keeping the program simple, use of data structures when possible (especially HashMaps), and a structured final review of the code.

e) Data Design (Extra Credit)

Identified data entities

- Portals
- PatientPortal
- NursePortal
- DoctorPortal

E-R diagram (you can use word or any software to draw the E-R diagram)



Relationships:

The one-to-one relationships between 'Login' and the Patient, Nurse, & Doctor Portal(s) means that only one <UserType>Portal entity is created per one 'Login' operation from Portals.

The one-to-many relationship between 'Portals' and 'Login', means that one Portals object can perform many Login(s). This is because the 'Portals' entity displays the Scenes/Pages allowing users to log into the Portal for their respective roles.

Credit Sheet

Team Member Name	Contributions
John Bostater	Final Astah Class Diagram, Phase-3 Video, Conclusion Writeup
Tristan Andrade	Phase-2 Video
Austin Mayhew	Testing Report, Phase-1 Video
Ryan Clark	Project Overview Video
Juan Rosas Jr.	Conclusion Video